11-768 HW2: Evaluating a Data-Visualization Agent

Zoe Xue
Andrew ID: zoex

Overview. This report follows one design thread: *every decision that has a mechanical answer is taken away from the judge model and made deterministic; the model is kept only for perception and language understanding, and even there its claims are verified against the pixels where physics allows.* Part 1 shows the released baseline is uninformative (macro-MCC 0.027) and diagnoses two mechanisms. Part 2 fixes them — a deterministic execution check and rubric-scoped, structurally parsed questions — reaching 0.448. Part 3 stress-tests that validator with five adversarial self-authored tasks (0.105 there), packages them as deterministic Harbor verifiers, and measures the exact boundary where deterministic verification stops. Part 4 turns those findings into a measured improvement campaign (frozen baselines, a repeat-noise floor, pre-registered candidates, audits of every changed prediction) whose product is the shipped validator:

validator	seed macro-MCC	authored macro-MCC
released baseline	0.0268	—
Part 2	0.4475	0.1047
shipped (Part 4)	**0.5002**	**0.6210**

Throughout, negative results are reported alongside positive ones: the one family that trades score for evidence soundness, the components that measurement rejected, and a reserved holdout on which the final validator does *not* beat its predecessor.

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

We ran the released baseline validator (`validator/baseline.py`) on all 110 seed runs and scored it against `seed_labels.json`:

Family	MCC	TP	TN	FP	FN
`execution_failure`	−0.0022	4	65	33	8
`wrong_data`	0.0000	0	73	0	25
`wrong_chart`	0.1096	3	27	0	68
`hard_to_read`	0.0000	0	59	0	39
macro-MCC	**0.0268**				

The baseline carries almost no signal, and its disagreements with the human labels concentrate in two places. On `execution_failure` it *over*-predicts: 33 of its 37 positive calls are false, while it misses 8 of the 12 genuinely failed runs. On the three content families it barely predicts at all

— and an important control-flow fact shapes that number: the baseline returns immediately after a positive execution verdict, so its 37 execution-positive runs never reach the content questions at all. Only 73 runs were actually asked the three content questions (219 questions, not 3×110), and across those the baseline produced five raw content positives (runs 059, 087, 096, 173, and 208, all `wrong_chart`), of which three are scored — 096 and 208 are excluded because their ground truth is a terminal `execution_failure`. Against 135 positive ground-truth content labels, `wrong_data` and `hard_to_read` are constant-NO (MCC 0 by definition) and `wrong_chart` keeps only the sliver of credit its three scored hits earn.

1.2 Validator Error Type 1: verdict-polarity confusion on the execution question

The baseline asks the negatively-framed compound question "Did the agent crash, exhaust its turns, or give up without producing a valid figure?" and takes any reply beginning with "YES" as an error. On 33 runs that produced a perfectly valid figure — among them **004, 022, 023, 034, 046, 062** — the recorded evidence *contradicts the recorded verdict*: the reply begins "Yes" and then continues, e.g. "The agent successfully completed the task. It created a Python script that generated a 2x2 grid of subplots …" (run 004) or "The agent successfully created the requested Sankey diagram…" (run 046). The model is answering the affirmative question it expected ("did the agent produce the figure?") rather than the negative one it was asked, and the prefix-based parser turns that polarity slip into a false error. The mirror image appears on the 8 missed true failures (**010, 053, 065, 089, 096, 168, 181, 208**): each of these run directories contains no `figure.png` at all, so the label is mechanical, yet none of them is predicted as an execution failure. Six have entirely empty predictions (e.g. 010, 053), and two (096, 208) instead carry a `wrong_chart` error — the judge went on to critique the design of a figure that does not exist. Because the baseline does not record negative replies, we cannot distinguish a genuine NO verdict from a reply whose phrasing simply failed the leading-"YES" parse; either way the family is decided by fragile surface form rather than by the artifact.

1.3 Validator Error Type 2: suppressed positives on the content families

For `wrong_data`, `wrong_chart`, and `hard_to_read`, the baseline asks one maximally broad question per family ("Does the figure fail to follow any part of the requested chart design?") with no definition of what counts, and again keeps only replies that begin with "YES". The result is near-total silence: 0/25, 3/71, and 0/39 scored true positives recovered. The misses are not subtle judgment calls; they include defects that are plainly observable in the figure and labeled with concrete evidence, on runs that *were* asked the content questions. In run **009**, the x tick labels were requested rotated 30 degrees, are drawn horizontal, and overlap into unreadability — `wrong_chart` and `hard_to_read`, neither flagged. In run **038**, the 'Non-smoker' series is drawn in the background colour and is invisible; in run **016** the requested $<0.5\%$ filter is ignored and the requested legend is missing; in run **030** the title reads "Shared axes" instead of the requested exact text; in run **213** the Men's column is plotted where the Women's column was requested. None of it is flagged. Three mechanisms combine here. The question gives the judge no rubric for what the family covers, so borderline YES cases default to a safe NO; the prefix parse discards any positive expressed in a different surface form; and error type 1 compounds the silence structurally — the 37 runs the baseline calls execution failures (run 004 among them, with its linear axis where log-base-3 was requested and its omitted cubing step) never reach the content questions at all, so a third of the dataset cannot produce a content positive no matter what the figure shows.

1.4 Proposed Validator Changes

For error type 1: take the model out of the loop entirely. `execution_failure` is decidable from the artifact: a run failed exactly when it left no decodable, non-empty `figure.png`. On the seed set this deterministic rule agrees with the ground truth on all 110 runs (12/12 positives, 98/98 negatives) and cannot suffer polarity or format errors.

For error type 2: replace each broad question with the assignment's own rubric for that family (the full sub-criteria list), instruct the judge to enumerate the task's explicit requirements and check them one by one against the figure and the plotting code, and require a fixed two-line answer tail (`VERDICT: YES/NO`, `EVIDENCE: …`) that is parsed structurally instead of by literal prefix, with a re-ask when the reply truncates before its verdict.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

Both changes are implemented in `validator/solution.py`.

Change 1: deterministic execution check. The baseline asks the VLM whether the agent "crashed, exhausted its turns, or gave up". `execution_failure`, however, is defined by the artifact, not the narrative: a run either left a decodable, non-empty `figure.png` or it did not. The improved validator decides this family without the model: missing, undecodable, or zero-sized figures are execution failures (evidence cites the final trajectory message), and everything else is not. On the seed set this rule reproduces the ground truth exactly (12/12 positives, 98/98 negatives). It is also model-agnostic by construction, which matters because the grading deployment uses the same fixed judge but a private set of runs.

Change 2: rubric-scoped questions with structured verdicts. The baseline asks one broad yes/no question per family (e.g. "does the figure fail to follow *any* part of the requested chart design?") and accepts the answer only if the reply literally starts with "YES". The improved validator sends, for each remaining family, the assignment's own sub-criteria for that family, an explicit instruction to check only what the task *requested* and to answer NO when the instructions left a choice open, and a checklist procedure (enumerate the explicit requirements, then compare each against the figure and the plotting code). Replies must end with `VERDICT: YES/NO` and `EVIDENCE: ...` lines, which are parsed structurally (with a robust fallback, and a re-ask when the reply was truncated before reaching a verdict). The model sees the instructions verbatim, the input files, a digest of the trajectory that preserves every executed command (the plotting code) while clipping prose and tool output, and the final figure. The baseline's reactive context-compaction and image downscaling are retained for oversized runs.

2.2 Seed-Set Performance vs. Baseline

	MCC		Improved (counts)			
Family	Baseline	Improved	TP	TN	FP	FN
`execution_failure`	−0.0022	**1.0000**	12	98	0	0
`wrong_data`	0.0000	0.3372	17	51	22	8
`wrong_chart`	0.1096	0.1387	20	23	4	51
`hard_to_read`	0.0000	0.3142	13	54	5	26
macro-MCC	0.0268	**0.4475**				

2.3 Qualitative Analysis of Targeted Error Types

Error type 1 (execution polarity) is eliminated. The deterministic check scores a perfect 1.0: all 33 baseline false positives (e.g. 004, 022, 046) and all 8 false negatives (e.g. 010, 053) are gone, and no new errors appear, because the decision no longer passes through the model at all.

Error type 2 (suppressed positives) is strongly mitigated but not solved. Positive predictions on the content families went from 5 raw / 3 scored to 81, recovering 17/25 `wrong_data`, 20/71 `wrong_chart`, and 13/39 `hard_to_read` true positives; every content-family MCC improved. Two causes contribute, and they should not be conflated: the rubric prompts and structural parsing raise the yield of the questions themselves, but change 1 also *ungates* the content questions — of the 37 runs the baseline short-circuited as execution failures, the 33 with valid figures are now all asked (the other four are genuine failures and remain terminal), so part of the recall gain is recovered coverage rather than better prompting. Where the questions fire, the evidence is genuinely grounded: on run **013** the prediction reads "uses the rainbow colormap instead of the required reversed plasma colormap" against the human label "the instructions forbid jet/rainbow/hsv, but the figure uses rainbow" (a scored `wrong_chart` true positive), and on run **016** it reads "includes versions with shares below 0.5% (e.g., Chrome 69.0 at 0.43, IE 9.0 at 0.44)" against the label's identical filter violation (a scored `wrong_data` true positive). Not every positive is this clean — runs 196 and 216 show the judge asserting defects (swapped group colours, a stacked-vs-grouped confusion) that the figures do not support, and both are scored false positives; hallucinated perceptual claims remain the main precision cost, taken up in Parts 3 and 4.

A representative remaining false positive. Run **013** is predicted `wrong_data` with the evidence "the task requires the reversed plasma colormap, but the figure uses the rainbow colormap…explicitly prohibited". The defect it describes is real, but a colormap choice is chart *style* — the ground truth files it (and we define it) under `wrong_chart` — so the same discovery is scored as a `wrong_data` false positive. Runs 030 and 051 show the same signature (title text and colormap violations reported as data errors). The judge finds the right defect and attributes it to the wrong family: the rubric's exclusion lists ("do NOT report style here") are not strong enough to stop a salient violation from leaking into whichever question notices it. A second false-positive mechanism appears in run **009** (`wrong_data`), where the model asserts that plotted matplotlib version numbers are wrong and versions are omitted; the human label records no data error, so this is a misreading of a dense timeline image — unverified perceptual claims stated as fact.

A representative remaining false negative. Run **030** is a `wrong_chart` false negative: the label's evidence is that the figure's title reads "Shared axes" instead of the requested exact text "Sharing x per column, y per row". The validator *did* report exactly this defect — but inside its `wrong_data` answer, so `wrong_chart` recorded a miss and `wrong_data` a false alarm. One family confusion thus costs two categories at once. More broadly, `wrong_chart` keeps 51 false negatives against only 27 negative runs; with truth this positive-heavy, the instruction to answer YES only when confident suppresses exactly the recall this family needs. What is missing from the validator is a cross-family arbitration step: verdicts are made independently per family, so nothing re-files a discovered defect into the family our definitions assign it to.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

Reading `validator/solution.py`, the two families still delegated to the VLM with the least corroborating signal are `wrong_data` and `hard_to_read`, which motivates two task classes.

Class 1: derived-value aggregation (`task_class` = "derived-value aggregation"). The validator judges plotted values from the rendered image plus the agent's code. When correctness hinges on a multi-step *derived* quantity — filter rows, then aggregate, then normalize or weight, then order by the result — the image alone cannot reveal a wrong number (a bar of height 79 looks as plausible as one of height 49), so the validator must effectively re-derive the answer from the code and input data. That is exactly the kind of arithmetic-over-context that a 30B judge is weak at, so near-miss aggregations (dropped filter, unweighted mean, raw counts instead of shares, order computed from the wrong quantity) may slip through.

Class 2: readability stress (`task_class` = "readability stress"). `hard_to_read` is judged purely by VLM perception of the rendered image. Subtle legibility defects — tick labels that collide or get clipped by the image edge, a legend sitting on top of a data series, neighbouring series in nearly identical colours — are known VLM blind spots, in both directions: a dense-but-legible chart invites false positives while genuinely overlapping text can be glossed over. Tasks that force many long tick labels or crowded legends probe this boundary directly.

3.2 Task Design

Five tasks (three in class 1, two in class 2); all data is synthetic and small enough to hand-verify, and every instruction pins the title, axis labels, ordering, and layout so correctness is human-reviewable.

- **`fine-totals`** (class 1; input `fines.csv`). Sum fines per library branch *excluding waived rows*, bars in descending order of total. The data is adversarial: with the waived filter the totals are {Eastside 53.5, Northgate 49.0, Harborview 42.5, Central 37.5}; without it they become {Northgate 79.0, Central 77.5, Eastside 55.75, Harborview 42.5} — three of the four totals and the required bar order change (Harborview alone keeps 42.5, having no waived rows), so a dropped filter is visible in both data and order.

- **`sensor-share`** (class 1; input `readings.csv`). Aggregate repeated per-site readings and plot each site's *percentage share* of the grand total (exactly 17.5/21.0/31.5/30.0, summing to 100) as a horizontal bar chart, alphabetical top-to-bottom, with formatted value labels. Traps: plotting raw sums instead of shares, and matplotlib's `barh` default of drawing the first category at the *bottom*.

- **`weighted-latency`** (class 1; data inline). Request-weighted mean latency per service (90.0, 80.0, 56.0, and 130.0 ms). The unweighted means (100/80/120/100) differ on three of four services, and the "search" service is a control where both agree.

- **`stage-attendance`** (class 2; data inline). Twelve bars with long stage names as tick labels, which must be rotated 45°, right-anchored, and fully visible on a canvas of at least 14×6 inches. Small default figures, missing rotation, or missing layout calls produce exactly the clipped/overlapping-label defects the class targets.

- **`city-temps`** (class 2; data inline). Six crossing temperature lines with pinned per-city colours and a legend that must sit *outside* the axes to the right without covering any line. Weak agents reliably leave the legend inside, where it occludes data — our very first generated run (`city-temps__qwen`) did exactly that.

3.3 Agent Runs and Labeling

Each of the five tasks was run once with each fixed agent (Qwen2.5-Coder-3B-Instruct, Ministral-3-14B-Instruct-2512, GLM-4.7-Flash) through the provided Modal agent runner: 15 runs, all under

runs/. Every run was reviewed against the assignment's evidence standard by reading the instructions, the trajectory (including the plotting code), and the rendered figure, and each label in labels.json cites its concrete evidence.

The outcome splits cleanly by agent strength. Three Qwen-3B runs (on fine-totals, sensor-share, and weighted-latency) produced no figure at all and ended with LimitsExceeded, each stuck in its own failure loop: the first repeatedly invoked sudo in a container that has none ("sudo: not found", returncode 127); the second died on a Python SyntaxError, a compound for statement placed after a semicolon in a one-liner; the third alternated between a Python SyntaxError and shell quoting errors (/bin/sh complaining of an unexpected parenthesis). That is three execution_failure labels, each citing its own trajectory. Qwen's other two runs produced figures with design errors: on stage-attendance it drew the correct bars but left numeric position ticks (0, 2, 4, …, 10) instead of the required stage names (wrong_chart), and on city-temps it placed the legend inside the axes against an explicit outside-right requirement (wrong_chart), where it covers the Farrow and Dunmore lines' January–February points (hard_to_read). The mid-size agents were stronger but fell into the designed ordering trap on sensor-share: both GLM and Ministral computed the percentage shares correctly and then let barh draw Alder at the bottom where the task demands it at the top (wrong_data, wrong order). Ministral's run also lacks the required "Site" y-axis label in the saved image — its script does call set_ylabel, but the label lands outside the saved region, and a required text element entirely outside the image counts as missing (wrong_chart). The remaining eight runs satisfied every requirement and are labeled acceptable.

Two labeling decisions deserve comment. First, the inverted sensor-share bar order is filed under wrong_data (not wrong_chart) because the family definitions place "wrong order: …sequences appear in a different order from the one requested" under data. Second, for city-temps__qwen the same legend produces two distinct errors — its placement violates an explicit design requirement, and, independently, it occludes data points — which is exactly the design-versus-legibility distinction the two families encode; the occlusion was confirmed by inspecting a crop of the legend region.

3.4 Validator Performance on New Tasks

The Part 2 validator, run unchanged on the 15 authored runs (the pre-iteration authored predictions file in output/):

Family	MCC	TP	TN	FP	FN	support (+/−)
execution_failure	1.0000	3	12	0	0	3/12
wrong_data	−0.3162	0	6	4	2	2/10
wrong_chart	−0.1741	0	8	1	3	3/9
hard_to_read	−0.0909	0	10	1	1	1/11
macro-MCC	**0.1047**					

With supports this small each error moves MCC a long way, but the failures are systematic, not noise, and land exactly in the two designed weakness classes. Three patterns account for every mistake. **(1) Hallucinated colour violations:** on all three city-temps runs the validator reported that the lines are "blue instead of tab:blue, orange instead of tab:orange" — but tab:blue *is* matplotlib's default blue, so the claim describes no visible difference. It poisoned the two clean runs (false wrong_data on GLM, false wrong_data + wrong_chart on Ministral) and displaced

the real errors on the Qwen run, which was predicted `wrong_data` instead of its true `wrong_chart` + `hard_to_read`. **(2) Missed order inversion:** on both `sensor-share` runs the alphabetical-from-top requirement is violated by `barh`'s bottom-up default, and the validator saw nothing wrong — the class-1 hypothesis (weak spatial/derived reasoning) confirmed on ordering rather than on values, which the agents computed correctly. **(3) Cross-family mis-filing with a perceptual hallucination:** on `stage-attendance__qwen` it correctly noticed the tick labels are not the stage names but filed the defect under `wrong_data`, and added a `hard_to_read` claim that "the full stage names are cut off" — there are no stage names in that figure at all. The agents' failures (three qwen `LimitsExceeded` runs, the legend and tick-label mistakes, the `barh` inversion) are weaknesses of the *agents*; every scoring mistake above is a weakness of the *validator*, and the three patterns directly motivate Part 4.

3.5 Harbor Packaging

Each of the five tasks is packaged under `harbor/tasks/<task_id>/`. The verifiers use three layers: S1-style artifact checks (a decodable, non-blank PNG of sane size; the input CSV unmodified, held to a sha256 literal; a plotting script left behind), an exact comparison of `plotted_values.json` against expected values derived by hand and written into the test as literals, and one re-execution of the delivered script in a scrubbed copy of `/app` (delivered artifacts are snapshotted first), which must exit 0 and reproduce a sidecar identical to the delivered one. All five tasks score 1.0 with `-a oracle`. The worked example was also run as required: oracle 1.0, `nop` 0.0, and two of its mutants (`sum_crates` 0.0; `illegible` 1.0).

Mutant scoring 0: `fine-totals/mutants/no_filter`. A plausible dropped-clause error: it sums every row, ignoring the waived filter. Reward 0. It passes every artifact and re-execution check and is rejected by exactly one assertion, `test_sidecar_declares_exactly_the_expected_values`, which reports the declared {Northgate 79.0, Central 77.5, Eastside 55.75, Harborview 42.5} against the hand-derived key.

Mutant scoring 1.0: `stage-attendance/mutants/illegible`. Every number is correct, the sidecar matches the key, and — deliberately — every structurally checkable clause of the instructions is satisfied: the canvas is the required 14×6 inches, the tick labels are the full stage names, rotated 45 degrees and right-anchored, and all text lies inside the saved image. What is wrong is purely perceptual: every text element is set at 2 pt, roughly three pixels tall at 100 dpi, unreadable by any person. Reward 1.0. Our verifier accepts it because its checks read file metadata, JSON values, and re-execution behaviour, never the rendered pixels; the S1 gates (image > 100 px on a side, > 1000 bytes, > 4 colours) are all cleared. We state the boundary carefully: this acceptance partly reflects checks we did not write — a verifier that instrumented matplotlib's artist tree, as `harbor/example` does, could read the font sizes off the figure and reject this particular mutant with a threshold. But legibility in general (labels that collide under rotation, low-contrast text, a legend occluding data) has no per-clause deterministic test, which is why the assignment's own accepted example works the same way. To make the contrast measured rather than asserted, we ran our validators on this mutant's figure (with its instructions and a minimal trajectory), twice each. The Part 2 validator flags `hard_to_read` — the right family — but with a fabricated mechanism (it calls the tick labels "clipped" when they are merely far too small). The final validator never accepts the mutant either, but on both repeats it files the problem elsewhere: a `wrong_data` claim fabricated from the axis text it cannot read, and a `wrong_chart` answer that in one repeat describes the true defect exactly ("font size 2 … the text is unreadable") while filing it in the wrong family. Its edge-verification stage, which requires a clipping claim to leave ink at the named image edge, correctly refuses the fabricated "clipped" mechanism — and with it, the lucky-family credit

the Part 2 validator earned. Text that is present, unclipped, and simply too small sits in the residual blind spot of the deterministic verifier and both VLM validators at once; we report that plainly rather than claiming the boundary crossed.

Could a deterministic per-task verifier decide each family? `execution_failure`: yes — file existence and PNG decodability are fully deterministic (our validator already decides it this way). `wrong_data`: for an *authored* task, largely yes — the sidecar plus hand-derived literals decides the plotted values, and re-execution ties them to the script; what it cannot see is a figure whose drawn geometry disagrees with the sidecar, which needs either savefig instrumentation (as in `harbor/example`) or perception. `wrong_chart`: only partially — structural facts (orientation, axis limits, tick text) are recoverable with matplotlib instrumentation, but only per-task, per-clause; each explicit design requirement needs its own assertion. `hard_to_read`: mostly no — individual proxies exist (a font-size threshold read off an instrumented artist tree would catch our own accepted mutant), but legibility as a property (labels colliding under rotation, contrast against a background, a legend occluding data) lives in the rendered pixels, admits no per-clause test, and is where the accepted mutant sits relative to the value-level checks we wrote.

Why a per-task verifier cannot be the grading mechanism. A Harbor verifier encodes the answer key for one known task: hand-derived literals, a schema for the sidecar, sha256 digests of the inputs, assertions matched to each clause of one instruction. The private grading set contains tasks we never saw, so there are no literals to write, no schema to pin, and no clause-by-clause assertions to prepare. Grading there requires a single function that generalizes across arbitrary instructions — deriving the expected chart from the task text, data, trajectory, and image at run time — which is precisely the validator, not a fixed test suite.

4 Part 4: Iterate on Your Design

4.1 Additional Improvements Motivated by Part 3

Why a campaign rather than one change. Part 3 exposed three failure patterns: colour hallucinations, missed order inversions, and cross-family mis-filing. Our first response was a composite change — deterministic colour filters plus a model arbitration pass — but a subsequent review found both components defective: the colour filter deleted valid findings (a genuine "the requested tab:blue series is red" loses its sentence to word co-occurrence; "the title is missing and the line colour is incorrect" loses the title defect too), and the arbitration parser could silently erase findings for families a truncated reply never answered. Patching by intuition had produced plausible-looking components that measurement rejected, so we re-ran Part 4 as a measured campaign instead.

Method. We froze the incumbent (predictions, scores, source hashes, a manifest of run/input hashes), added optional per-call diagnostics (disabled by default; reads no labels; changes no decisions), built a fixed 39-run diagnostic subset enriched with hard cases from both released and authored runs, and measured the incumbent twice on it and twice on the authored set, giving a repeat-noise floor of about ±0.02–0.03 macro-MCC that every promotion is judged against. Each candidate was pre-registered (hypothesis and plausible regression written before running), gated on local regression tests, evaluated on the subset, and audited finding by finding; the full ledger ships in `experiments/LEDGER.md`. The diagnostics answered the first question directly: of ∼38 content-family misses on the subset, 34–35 were never detected by the first-stage questions — recall was bounded by *detection*, not by filtering or arbitration — which set the candidate order below.

Candidates and decisions. (Subset macro-MCC; incumbent repeats 0.268/0.289.)

candidate (one component each)	subset	decision
remove destructive colour filter; arbiter may not invent verdicts	0.269	adopt (correctness)
checklist `wrong_chart` prompting	0.370	adopt
readability prompt alone	0.379	adopt only with edge check
deterministic mapping replaces model arbiter	0.432	adopt
composed + edge verification + fact layer	**0.610**	champion
+ dual-filing dedup (post-champion)	replay	adopt
+ code-literal contrast check (post-champion)	audit	adopt

Adopted changes, in pipeline order.

1. **Correctness repairs.** The colour filter is disconnected (four of five required-preservation regression cases fail against it), and the arbiter can no longer emit a verdict for a family no reviewer proposed with fabricated "confirmed by arbitration" evidence. Subset score unchanged within noise — kept because verified correctness repairs are not traded for score.

2. **Checklist `wrong_chart` prompting.** The broad "check each requirement" instruction becomes an enumerated checklist (at most 12 items: quote, expected, observed, source image/code, status pass/fail/unknown) with an explicit image-over-code rule — a property set in code but absent from the saved image is absent. Subset `wrong_chart` $-0.045 \rightarrow +0.279$ (TP 5→15), recovering legend, axis-label, orientation, axis-limit, and colorbar defects, including `sensor-share__ministral`'s off-canvas label.

3. **Readability prompting with deterministic edge verification.** `hard_to_read` no longer answers under the "explicit requirement" confidence gate (legibility defects are rarely explicit clauses); it walks element groups with CHECK lines and must name the element, its location, and a visible mechanism. Alone this recovered real occlusion/overlap/invisibility defects but amplified "partially cut off" fabrications (11 new false alarms); an edge-ink audit showed only one clipping claim in the whole subset run was physically real. A deterministic verifier therefore strikes any clipping claim whose named image edge holds no ink. Replayed on the same run: `hard_to_read` $+0.107 \rightarrow +0.444$, false positives 16→1.

4. **Deterministic family mapping replaces model arbitration.** An ablation replaying *identical* recorded findings under three configurations scored: direct +0.378; deterministically re-filed +0.436; model-arbitrated +0.370. The arbiter erased more true positives than it repaired, so findings are re-filed by rule (drawn order → `wrong_data`; missing/wrong title, label, legend, colormap, scale → `wrong_chart`; partial clipping → `hard_to_read`) and the arbitration call is gone — one model call per positive run saved.

5. **Deterministic fact layer.** An explicit DPI request is compared with the PNG's own metadata (fires on exactly the three true violations across all 125 development runs; silent on the twelve compliant DPI-mentioning runs), and literal number series in the instructions are compared with literal lists in the last successful figure-writing command (fires on exactly one run, 077's re-ordered series; sorting requests suppress the order rule). Both audited offline to zero false positives before inclusion.

6. **Dual-filing dedup.** One real defect reported by two family questions at once costs a guaranteed false positive in one of them ("sd instead of sem" filed as both `wrong_data` and

`wrong_chart`). Near-duplicate cross-family errors — a similarity gate calibrated on 75 real evidence pairs, with genuinely distinct coexisting defects sitting safely below it — collapse toward the family their combined language names.

7. **Code-literal contrast check.** Several `hard_to_read` truths share one mechanism: an element drawn in a colour too close to the background, with the colour sitting in the agent's code as a literal. Colour kwargs from every successful command (excluding edge colours, legend-proxy handles, unlabeled reference lines, translucent fills, masking patches, and dark-canvas runs) are compared against the rendered corner background; a WCAG-style contrast ratio below 2.5:1 files `hard_to_read`. After three audit iterations it fires on 8 runs across all 125 development runs — every one a `hard_to_read` truth — with zero false positives. Replaying both of these last two components over the three fixed seed sweeps of the pre-revision champion raised macro-MCC by a consistent +0.045 to +0.055 (about twice the repeat-noise floor), with `hard_to_read` moving from 0.08 to 0.26–0.29.

4.2 Final Evaluation on Seed and New Tasks

The designated final sweeps of the shipped validator, against the Part 2 version's designated sweeps:

	Seed runs		Authored runs	
Family	Part 2	Final	Part 2	Final
`execution_failure`	1.0000	1.0000	1.0000	1.0000
`wrong_data`	0.3372	**0.3553**	−0.3162	**0.0756**
`wrong_chart`	0.1387	**0.3061**	−0.1741	**0.4082**
`hard_to_read`	0.3142	**0.3395**	−0.0909	**1.0000**
macro-MCC	0.4475	**0.5002**	0.1047	**0.6210**

Seed `wrong_chart` recall rose from 23/71 to 50/71 true positives, and `hard_to_read` — the family Part 4's first revision traded away — recovered to 0.3395 (14/39 true positives, 5 false positives) once the code-literal contrast check joined the deterministic layer. Repeat variation is reported in full rather than summarized away: before selection, finalist and incumbent were each run twice more, fresh and interleaved under matched serving:

macro-MCC	seed		authored	
	r1	r2	r1	r2
Part 2 incumbent	0.4389	0.4700	0.3552	0.2808
pre-revision champion	**0.4800**	**0.4939**	**0.6787**	**0.6529**
shipped (designated / repeat)	0.5002	0.5093	0.6210	0.6416

On seed, the champion's worse repeat beats the incumbent's better one. The two final deterministic components (dedup, contrast) were additionally measured by exact replay over the three fixed pre-revision seed sweeps — 0.4800/0.4931/0.4939 → 0.5338/0.5367/0.5411, a consistent +0.045 to +0.055 — which isolates their effect from the fresh-sweep variance of the model-judged families.

Two results are reported against ourselves. First, the `hard_to_read` story is a deliberate two-step trade. The edge-verification stage initially dropped the family to 0.08: we investigated every lost true positive across the comparison sweeps, and all twelve were fabricated clipping claims that happened to land on runs whose real defect is a different mechanism (overlapping tick labels, an

invisible series, upside-down text, a legend covering tick labels) — unsound credit, removed on purpose. The code-literal contrast check then rebuilt recall on verifiable evidence (008's `#f4f4f4` labels, 200's white scatter series), landing at 0.3395 — above the Part 2 version's 0.3142, now with every clipping or contrast claim physically checked. The remaining 25 misses are non-clipping mechanisms the judge still overlooks (e.g. 009's overlapping tick labels). Second, before tuning we built and reserved a fresh 26-run validation suite (12 correct/faulty pairs plus two execution failures; constructed and visually verified by us, not human-reviewed course labels; provenance and scores in `experiments/`) and evaluated it exactly once after selecting the champion: incumbent macro +0.7248, champion +0.7108 — statistically a tie inside the noise floor, with the champion perfect on readability (4/4, no false positives) and more false-positive-prone on clean charts elsewhere. The dedup component was subsequently designed partly from that suite's observed errors, so it is now development data and *no untouched holdout remains*; the development-set gains do not demonstrate private-set superiority, and the defensible claim stays: comparable-or-better classification with substantially sounder evidence, at one model call less per positive run. Our engineering stretch target of 0.60 seed macro-MCC was not reached (designated 0.5002; replay band 0.534–0.541); the subset result (0.610) shows the pipeline can get there on hard cases, and the open `hard_to_read` and `wrong_chart` recall gaps are the identified remainder.

4.3 Qualitative Analysis of Self-Authored Tasks

On the designated authored sweep the shipped validator scores 0.6210, catching `city-temps__qwen` exactly (`wrong_chart` + `hard_to_read`, the legend-over-lines mechanism named correctly), `sensor-share__ministral`'s order inversion as `wrong_data`, and `stage-attendance__qwen`'s missing stage names; the twin inversion on `sensor-share__glm` was caught in the previous designated sweep but missed in this one — the run-to-run instability of that detection, noted below, cuts both ways.

A false positive. `city-temps__ministral` (a correct run) is again flagged, this sweep as `wrong_data`: "Brinport is shown in orange instead of tab:orange (which is correct), but the main [violation]…" — evidence that concedes its own claim. The trajectory contains `'Brinport': 'tab:orange',` and the drawn lines match it; the claim asserts a difference no pixels can show. This is the known cost of removing the colour filter: that filter was measurably destructive (it also deleted genuine colour defects), so we took the principled trade and this hallucination channel stays open. The second residual FP channel is re-derivation: `fine-totals__ministral` is accused of breaking the descending order its own code (`sorted(totals, key=..., reverse=True)`) demonstrably satisfies — the judge re-ranks the four close totals in its head and trusts its arithmetic over the image.

A false negative. `sensor-share__ministral`'s `wrong_chart` label is still missed. The script calls `ax.set_ylabel('Site')`, but the label lies entirely outside the saved image; a judge that weighs the code sees the requirement satisfied. The final validator can catch this mechanism class — its checklist's image-over-code rule recovered this very run on the diagnostic subset — but detection is unstable run to run, and the designated sweep missed it (that sweep ran without diagnostics, so we cannot say which checklist line failed). Noticing an *absence* in the image against a *presence* in the code remains the hardest case for the judge.

4.4 Further Changes

Everything beyond the adopted changes is infrastructure, shipped for reproducibility rather than score: the optional diagnostics layer (`HW2_DIAG_DIR`), the frozen Part 2 validator

(validator/solution_part2.py; the pre-iteration numbers regenerate with --solution validator.solution_part2), the experiment ledger and per-candidate frozen sources under experiments/ (e00 incumbent freeze, candidate shas, the arbitration ablation, the reserved-suite builder and its labels), and regression tests covering the filtering and malformed-reply cases that started the campaign. Cost: the shipped validator makes 3 judge calls per valid-figure run (plus at most one bounded truncation re-ask each; no arbitration call), and 0 calls for missing-figure runs; the full 110-run seed sweep completes in roughly 35–45 minutes on the course deployment. Known open limitations, measured and not patched around: residual dual-filing (the dedup stage collapses the pattern only above its calibrated similarity gate; a sub-gate pair survives in the holdout's scatter-styled run), re-derivation and misbinding hallucinations on correct charts (now the dominant false-positive channel — e.g. the fine-totals order re-derivations in the designated sweep), the non-clipping hard_to_read recall gap (25 seed misses, e.g. 009's overlapping tick labels), and tiny-but-unclipped text, which evades the deterministic verifier and both VLM validators alike. The natural next candidate — targeted crop verification for proposed legibility defects — is specified in the ledger but was not pursued after the campaign's stopping review.

4.5 Reproducibility

Every number in this report regenerates from the submission. The Part 1 table: run validator.baseline through validator.runner on artifacts/ and score against seed_labels.json. The Part 2 tables: the frozen module validator/solution_part2.py, selected with the runner's --solution flag; its designated predictions are output/pre_iteration_*.json. Parts 3–4: the shipped validator/solution.py; designated predictions are output/final_seed_predictions.json and output/authored_predictions.json, scored with workflow score (against labels.json for authored runs). Harbor rewards: workflow check-submission re-runs every oracle and both mutants. The experiment ledger — per-candidate hypotheses, frozen sources with SHA-256s, subset scores, the arbitration ablation, and the reserved-suite builder and labels — is under experiments/; local gates run offline via experiments/regression_tests.py and experiments/arbitration_ablation.py. Judged sweeps are nondeterministic at the margin (batched serving), so comparisons should use the reported repeat bands rather than single sweeps.